

Résolution numériques des équations non linéaires

1.Introduction

Calculer les zéros d'une fonction f réelle (c'est-à-dire les racines d'une équation $f(x) = 0$) est un problème que l'on rencontre très souvent en Calcul Scientifique. En général, cette tâche ne peut être effectuée en un nombre fini d'opérations. Par exemple, nous avons vu au paragraphe 1.5.1 qu'il n'existait pas de formule explicite donnant les racines d'un polynôme quelconque de degré supérieur à 4. La situation est bien sûr encore plus complexe quand f n'est pas un polynôme. Pour résoudre le problème, on utilise donc des méthodes itératives : partant d'une ou plusieurs valeurs initiales, on construit une suite de valeurs (x_k) qui, si tout se passe bien, converge vers un zéro α de la fonction f considérée. Nous débuterons ce chapitre avec quelques problèmes simples et concrets qui donnent lieu à des équations non linéaires. Diverses méthodes numériques seront alors présentées, puis utilisées pour résoudre ces problèmes. Cette démarche sera également adoptée dans les chapitres suivants.

2. Séparation des zéros

3. Vitesse de convergence de suite

La résolution numérique d'une équation non linéaire de la forme $f(x) = 0$ consiste à construire une suite (x_n) qui converge vers x^* l'unique solution de notre problème (on suppose pour le moment que l'équation admet une unique solution).

Définition 2.1. Ordre de convergence d'une suite.

Soit (x_k) une suite réelle, et x^* un réel.

— On dit que (x_n) converge linéairement (ou à l'ordre 1) vers x^* s'il existe des constantes $C > 0$ et $\alpha \in]0, 1[$ telles qu'à partir d'un certain rang, on ait :

$$|x_n - x^*| \leq C\alpha^n. \quad (1.2)$$

— On note r la borne inférieure des α qui satisfont la condition ci-dessus, et r est appelé taux de convergence linéaire de la suite.

— Si $r = 0$, on dit que la convergence est superlinéaire.

— Enfin, s'il existe des constantes $C > 0$, $\alpha \in]0, 1[$ et $\beta > 1$ telles qu'à partir d'un certain rang, on ait :

$$|x_n - x^*| \leq C\alpha^{\beta^n}. \quad (1.3)$$

alors on dit que la convergence est d'ordre au moins β . La borne supérieure de ces β est appelé l'ordre de convergence de la suite.

L'ordre de convergence est lié à la vitesse à laquelle le nombre de chiffres exacts augmente dans l'écriture décimale de (x_n) (par rapport aux décimales de x^*). Dans le cas de la convergence linéaire, le nombre de décimales s'accroît d'un facteur constant à chaque étape (ce facteur dépendant de r : plus r est petit, et plus le nombre de décimales exactes ajoutées à chaque étape est grand). Dans le cas de convergence d'ordre β avec $\beta > 1$, le nombre de décimales exactes est multiplié par un facteur β à chaque étape.

Pour comparer deux méthodes, on comparera donc d'abord leur ordre de convergence (par abus de langage, on appelle ordre de convergence d'une méthode l'ordre de convergence des suites qu'elle génère). Si les deux sont linéaires, alors on peut comparer leur taux de convergence linéaire pour mesurer leur rapidité de convergence (plus le taux est petit, plus la vitesse est grande).

4. Méthode de dichotomie

4.1. Présentation de la méthode

La méthode de dichotomie, basée sur le théorème de la valeur intermédiaire pour les fonctions continues est, en quelque sorte, ce nous pourrions appeler une méthode de calcul par tâtonnements organisés. Si la fonction $f(x)$ est continue sur l'intervalle $[a, b]$ et telle que $f(a).f(b) < 0$ il y a au moins une racine dans (a, b) . La moitié de l'intervalle qui contient la racine sert de nouvel intervalle pour l'itération suivante. La méthode de dichotomie génère donc une suite d'intervalles emboîtés $[a_n, b_n]$ telle que pour tout n : $f(a_n).f(b_n) < 0$.

4.2. Résultat de convergence

Proposition 3.1

On suppose que f est une fonction continue sur un intervalle I et qu'il existe des points de I , a_0 et b_0 avec $a_0 < b_0$ tels que $f(a_0) < 0 < f(b_0)$. La méthode de dichotomie est la suivante : par récurrence, on pose $c_n = (a_n + b_n)/2$ le point milieu et on définit ensuite $(a_{n+1}, b_{n+1}) = (a_n, c_n)$, si $f(a_n) * f(c_n) < 0$ et $(a_{n+1}, b_{n+1}) = (c_n, b_n)$, sinon. Alors les suites (a_n) ; (b_n) ; (c_n) convergent vers une limite x^* qui vérifie $f(x^*) = 0$, et la convergence est linéaire. Si la suite n'est pas constante à partir d'un certain rang (autrement dit si on n'est jamais dans le cas $f(c_n) = 0$), alors le taux de convergence linéaire est $1/2$.

4.4. Evaluation de la méthode

3.4.1. Les avantages

1. La convergence certaine de l'algorithme vers la racine cherchée si f est continue sur l'intervalle $[a, b]$ dans lequel on a réussi à isoler une racine unique ;
2. La possibilité de déterminer a priori le nombre d'itérations à effectuer pour obtenir la racine voulue avec une précision donnée. En effet, on part d'un intervalle connu $[a, b]$ et on génère des intervalles dont la longueur vaut la moitié de celle de l'intervalle précédent. Ainsi, après n itérations, l'intervalle qui encadre la racine cherchée est de longueur $\frac{(b-a)}{2^n}$ et si l'on veut que l'erreur absolue commise sur le résultat soit inférieure à une quantité ϵ prédéfinie, il suffira de faire n itérations avec $n > \frac{\log(b-a)}{\log(2)}$.
3. Un seul calcul de fonction à chaque itération.

3.4.2. Les inconvénients

1. Vitesse de convergence linéaire, donc lente.
2. Sensible aux erreurs d'arrondi ; exemple : la fonction $f(x) = e^x - 1 - x - x^2/2$ s'annule théoriquement en $x = 0$ seulement. Numériquement, elle change de signe un grand nombre de fois autour de $x = 0$.
3. les contraintes imposées par les hypothèses de départ dont, principalement, le fait qu'il faille déterminer un intervalle qui encadre une seule racine et dans lequel la fonction étudiée change de signe. .

4.3. Programmation de la méthode

Le code Python:

Dans cette section, nous essayerons de résoudre numériquement l'équation $x^2 - 2 = 0$ afin de trouver une approximation de $\sqrt{2}$.

Il existe en fait une commande spéciale à IPython : `%pylab` qui permet de charger les outils de `numpy` (pour le calcul scientifique) et de `matplotlib` (pour les graphiques). Le mot-clé `inline` permet d'afficher les graphiques directement dans le notebook (si on le supprime les graphiques seront ouverts dans une nouvelle fenêtre), et vous permettra ainsi de renvoyer directement le TP en fin de séance avec les graphiques que vous aurez produits. Pour les TPs suivants, on commencera donc **TOUJOURS** par cette ligne-là que l'on n'oubliera pas de valider (et de revalider dès que l'on redémarre le *Kernel*).

In [124]:

```
%pylab inline
```

Populating the interactive namespace from numpy and matplotlib

In [42]:

```
def f(x):
    return x**2-2
```

Voilà une version d'une fonction de la méthode de dichotomie programmée avec la boucle `for`. On remarque qu'on a une condition d'arrêt lorsque a se rapproche de b ($b - a < tol$). Mais dans le cas de non convergence l'algorithme s'arrête on N_{max} itération.

In [4]:

```
def dichotomie(f,a,b,Nmax,tol):
    for i in range(Nmax):
        c = (a+b)/2
        if b-a<tol:
            return c
        if f(a)*f(c) <= 0:
            b = c
        else:
            a = c
    return c
```

On fait appel à la fonction pour résoudre l'équation $x^2 - 2 = 0$.

In [5]:

```
x=dichotomie(f,0.,2.,Nmax=10000,tol=10**-6)
```

In [6]:

```
x
```

Out[6]:

```
1.4142136573791504
```

A ce stade on modifie notre fonction `dichotomie` de telle façon qu'on stocke tous les termes de la suite dans une

liste L.

In [16]:

```
def dichotomie(f,a,b,Nmax,tol):
    L=[]
    for i in range(Nmax):
        c = (a+b)/2
        L.append(c)
        if b-a<tol:
            break
        if f(a)*f(c) <= 0:
            b = c
        else:
            a = c
        print('{0:10d} | {1: 12.10f} | '.format(i, c))
```

In [17]:

```
dichotomie(f,0.,20.,Nmax=10000,tol=10**-8)
```

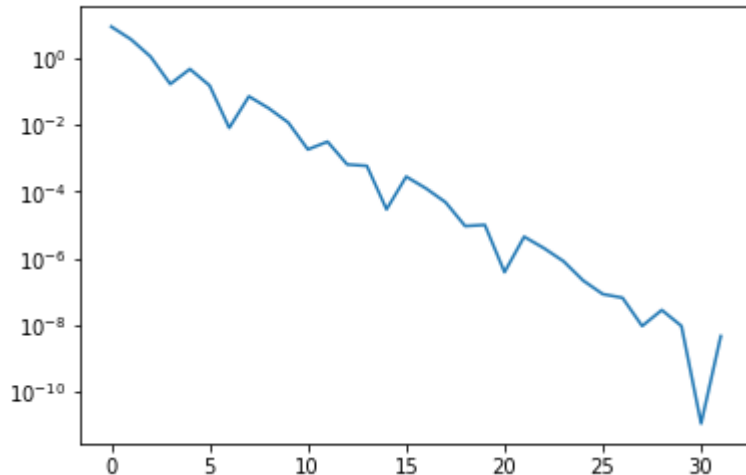
```
0 | 10.0000000000 |
1 | 5.0000000000 |
2 | 2.5000000000 |
3 | 1.2500000000 |
4 | 1.8750000000 |
5 | 1.5625000000 |
6 | 1.4062500000 |
7 | 1.4843750000 |
8 | 1.4453125000 |
9 | 1.4257812500 |
10 | 1.4160156250 |
11 | 1.4111328125 |
12 | 1.4135742188 |
13 | 1.4147949219 |
14 | 1.4141845703 |
15 | 1.4144897461 |
16 | 1.4143371582 |
17 | 1.4142608643 |
18 | 1.4142227173 |
19 | 1.4142036438 |
20 | 1.4142131805 |
21 | 1.4142179489 |
22 | 1.4142155647 |
23 | 1.4142143726 |
24 | 1.4142137766 |
25 | 1.4142134786 |
26 | 1.4142136276 |
27 | 1.4142135531 |
28 | 1.4142135903 |
29 | 1.4142135717 |
30 | 1.4142135624 |
```

On peut tracer $|c_n - x^*|$ de deux manières : soit en connaissant la valeur théorique $\sqrt{2}$, soit en approximant x_∞ par le dernier point calculé. On peut utiliser la fonction `abs` qui s'applique sur des tableaux de type `array` (donc il faut transformer la liste en `array` si on a généré la suite par une liste, on peut aussi directement remplir un tableau vu qu'on connaît sa taille a priori).

On doit observer une droite en échelle semi-logarithmique. Pour mieux le voir, on peut tracer $|\ln |c_n - x^*||$ en échelle semi-log et voir des droites. Mais elles sont tronquées du fait que l'on atteint rapidement la précision machine et admettent quelques perturbations dues aux erreurs d'arrondis.

In [53]:

```
semilogy(abs(array(1-sqrt(2))))
show() # enlever cette commande et voir qu'est ce qui se passe ?
```



En principe, puisque la convergence de la méthode de dichotomie est linéaire la courbe doit suivre une droite affine lorsque la suite (c_n) se rapproche de $\sqrt{2}$. Les perturbations qu'on remarque sur la figure sont dues aux erreurs d'arrondis.

La fonction prédéfinie dans Python "bisect"

La méthode de dichotomie est déjà programmée en Python



In [5]:

```
def f(x):
    return x**2-2
```

In [3]:

```
from scipy.optimize import *
x=bisect(f,1,2)
```

In [4]:

```
x
```

Out[4]:

```
1.4142135623715149
```

4. Méthode de Newton

La seule information utilisée par la méthode de dichotomie est le signe de la fonction f aux extrémités des sous-intervalles. Dans le cas où f est différentiable, on peut construire une méthode plus efficace en exploitant les valeurs de f et de ses dérivées. En partant de l'équation de la tangente à la courbe $(x, f(x))$ au point x^k ,

$$y(x) = f(x^k) + f'(x^k)(x - x^k),$$

et en faisant comme si x^{k+1} vérifiait $y(x^{k+1}) = 0$, on obtient:

$$x^{k+1} = x^k - \frac{f(x^k)}{f'(x^k)}.$$

4.1. Tests d'arrêt pour les itérations de Newton

En théorie, une méthode de Newton convergente ne retourne le zéro α qu'après une infinité d'itérations. En pratique, on recherche une approximation de α avec une certaine tolérance ϵ . Ainsi, on peut interrompre la méthode à la première itération k_{min} pour laquelle on a l'inégalité suivante

$$e_{n_{min}} = |x_{n_{min}} - x^*| < \epsilon$$

Ceci est un test sur l'erreur. Malheureusement, comme l'erreur est elle-même inconnue, on doit la remplacer par un estimateur d'erreur, c'est-à-dire, une quantité qui peut être facilement calculée et grâce à laquelle on peut estimer l'erreur réelle. La différence entre deux itérées successives fournit un estimateur d'erreur correct pour la méthode de Newton. Cela signifie que l'on peut interrompre les itérations à l'étape k_{min} telle que

$$|x_{n_{min}} - x_{n_{min}-1}| < \epsilon$$

Ceci est un test sur l'incrément.

On pourrait utiliser alternativement un test sur le résidu à l'itération k , $r(k) = f(x_k)$ (remarquer que le résidu est nul quand x_k est un zéro de la fonction f). Plus précisément, on pourrait arrêter les itérations à l'étape k_{min} pour laquelle

$$r_{n_{min}} = |f(x_{n_{min}})| < \epsilon$$

Le test sur le résidu n'est satisfaisant que quand $|f'(x)|$ est proche de 1 dans un voisinage I_α du zéro α .

Autrement, il a tendance à surestimer l'erreur si $|f'(x)| \ll 1$ pour $x \in I_\alpha$ et à la sous-estimer si $|f'(x)| \gg 1$.

4.2. Résultats de convergence

Théorème 4.1 (Convergence locale)

Soit $I \subset \mathbb{R}$ intervalle de $\mathbb{R}$ et $f : I \rightarrow \mathbb{R}$ de classe C^2 : On suppose que f s'annule en un point a à l'intérieur de I et que $f'(a) \neq 0$. Alors, la suite $(x_n)_{n \in \mathbb{N}}$ définie par

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}.$$

converge quadratiquement vers a pour x_0 assez proche de a .

4.3. Programmation de la méthode en Python

Nous présentons divers versions de codes de la méthode de Newton programmée en python. La première version qu'on va présenter est celle où la dérivée est calculée en à travers la fonction prédéfinie dans Python "derivative" du module "misc" contenu dans le package "scipy".

In [8]:

```

from scipy import misc

def NewtonsMethod(f, x, tolerance=10**-8):
    while True:
        x1 = x - f(x) / misc.derivative(f, x)
        t = abs(x1 - x)
        if t < tolerance:
            break
        x = x1
    return x

```

La fonction "NewtonsMethod" prend en entrée la fonction f , le point initial x et la tolérance et retourne la solution. On peut modifier cette fonction d'une telle façon qu'elle retourne une liste qui contient tous les éléments de la liste.

In [26]:

```

from scipy import misc

def NewtonsMethod(f, x, tolerance=10**-8):
    L=[]
    n=0
    while True:
        n+=1
        L.append(x)
        x1 = x - f(x) / misc.derivative(f, x)
        t = abs(x1 - x)
        if t < tolerance:
            break
        x = x1
        print('{0:10d} | {1: 12.10f} | '.format(n, x))
    return(L)

```

In [29]:

```
L=NewtonsMethod(f, x=.5, tolerance=10**-15)
```

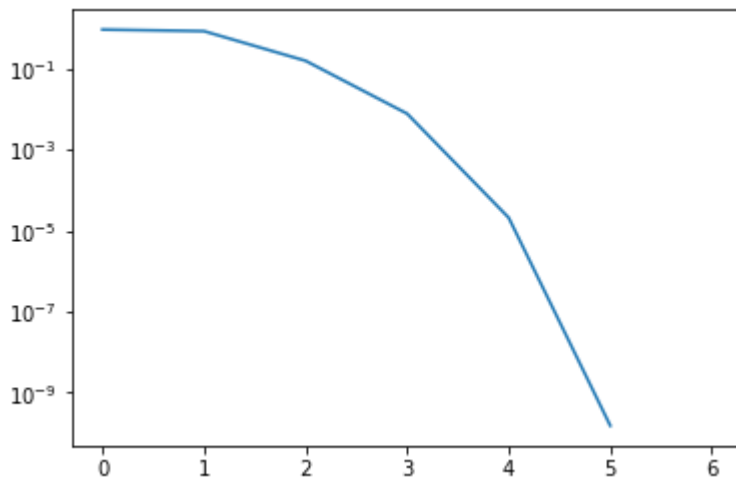
```

1 | 2.2500000000 |
2 | 1.5694444444 |
3 | 1.4218903638 |
4 | 1.4142342859 |
5 | 1.4142135625 |
6 | 1.4142135624 |

```

In [31]:

```
semilogy(abs(L-sqrt(2)))
show()
```



A l'échelle semilogarithmique, on n'obtient pas une droite de pente négative car la convergence n'est pas linéaire (d'ordre 2).

4.4. Méthode de Newton prédéfinie dans Python

On peut utiliser la fonction prédéfinie `newton` de `scipy.optimize`. Ainsi, `resol.newton(f, x0)` détermine une racine par la méthode de Newton en partant de x_0 .

In [9]:

```
% pylab inline
```

Populating the interactive namespace from numpy and matplotlib

```
C:\ProgramData\Anaconda3\lib\site-packages\IPython\core\magics\pylab.py:161:
UserWarning: pylab import has clobbered these variables: ['fmin', 'f', 'zero
s']
`%matplotlib` prevents importing * from pylab and numpy
"\n`%matplotlib` prevents importing * from pylab and numpy"
```

In [11]:

```
def f1(x):
    return x**2-2
```

In [12]:

```
def df1(x):
    return 2*x
```

In [13]:

```
from scipy.optimize import *
x=newton(f1,0.5,df1)
```


In [14]:

```
abs(x-sqrt(2))
```

Out[14]:

0.0

In [16]:

```
from scipy.optimize import *
y=newton(f1,0.5)
```

In [17]:

```
abs(y-sqrt(2))
```

Out[17]:

2.2204460492503131e-16

On a utiliser la méthode de Newton de deux manières différentes, la première façon consiste à donner l'expression explicite de f' tant dis que dans la deuxième manière nous contontons à donner en entré seulement f et x_0 . Dans ce cas, la fonction Newton prédéfinie dans Python sera obligés de trouver une approximation de f' et par conséquent, le résultat donné par la méthode de Newton est légèrement perturbé (voir le test ci-dessus).

4.5. Evaluation de la méthode de Newton

4.5.1. Les avantages

1. Converge rapidement quand elle converge (ce qui compense largement le dernier inconvénient).
2. Relativement stable et peu sensible aux erreurs d'arrondis si $f'(x^*)$ n'est pas trop petit.

4.5.2. Les inconvénients :

1. Peut diverger ou converger vers un autre zéro que celui cherché si la donnée initiale est mal choisie.
2. Nécessite le calcul de la dérivée d'une fonction, ce qui est numériquement difficile si on ne la connaît pas explicitement chaque étape nécessite deux évaluations de fonctions.

5. Méthode de la sécante

5.1. Présentation de la méthode

Bien que la méthode de Newton est très utilisée dans la pratique, son principal inconvénient vient du fait de l'utilisation à chaque itération de la dérivée. Quand la fonction f n'est pas définie explicitement, car en calcul numérique, les fonctions sont généralement représentées par des vecteurs et non pas par leurs expressions explicites. Par exemple: Une fonction f définie sur un intervalle $[a, b]$ est approximée par un vecteur de la forme $[f(x_0), \dots, f(x_n)]$ où les (x_i) , $0 \leq i \leq n$ est une discrétisation de l'intervalle $[a, b]$. Autrement dit on a accès seulement aux (x_i) , $0 \leq i \leq n$ et aux $(f(x_i))$, $0 \leq i \leq n$. C'est pourquoi nous allons voir maintenant la méthode de la sécante qui n'utilise pas la dérivée de f . L'idée de cette méthode est d'approcher la dérivée $f'(x_i)$ par une différence dévisée $\frac{f(x_i) - f(x_{i-1})}{x_i - x_{i-1}}$. L'itération de la méthode de la sécante s'écrit donc

$$x_{i+1} = x_i - \frac{f(x_i)(x_i - x_{i-1})}{f(x_i) - f(x_{i-1})}.$$

L'interprétation géométrique de la méthode de la sécante est comme suit: Soient x_0 et x_1 deux approximations de la racine r d'une équation $f(x) = 0$. Si, dans un voisinage de l'intervalle $[x_0, x_1]$ on remplace le graphe de $f(x)$ par une droite qui joint les points $(x_0, f(x_0))$ et $(x_1, f(x_1))$ on peut espérer trouver une nouvelle approximation de la racine cherchée en assimilant celle-ci au point x_2 où la droite coupe l'axe des x .

5.2. Théorème de convergence

Théorème 5.1.

Soit f deux fois continument différentiable sur un intervalle ouvert de centre x^* vérifiant : $f(x^*) = 0, f'(x^*) \neq 0$. Alors, si x_0, x_1 sont choisis assez près de x^* , l'algorithme de la sécante défini par :

$$x_{n+1} = x_n - \frac{f(x_n)(x_n - x_{n-1})}{f(x_n) - f(x_{n-1})}.$$

$\forall n \geq 1$. converge vers x^* et la convergence est au moins d'ordre $p = \frac{\sqrt{5}+1}{2} = 1,618...$ (nombre d'or).

5.3. Programmation de la méthode

In [108]:

```
def g(x):
    return x**2-2
```

In [110]:

```
def secante(x0,x1,f, epsilon = 10**(-12),maxiter = 30):
    t = x0
    u = x1
    n = 0
    L=[x0]
    v = u - f(u)*(u-t)/(f(u)-f(t))
    while abs(v-u) > epsilon:
        L.append(u)
        n = n+1
        if n > maxiter:
            chn = "Echec après {} itérations.".format(maxiter)
            raise RuntimeError(chn)
        u, v = v, v-f(v)*(v-u)/(f(v)-f(u))
    #print("La solution par la sécante de la fonction ",f.__name__," est: ",v," réalisé
    print('{0:10d} | {1: 12.10f} | '.format(n, u))
```

In [114]:

```
secante(1.7, 1.5,g, epsilon = 10**(-15),maxiter = 30)
```

```
1 | 1.4218750000 |
2 | 1.4144385027 |
3 | 1.4142141700 |
4 | 1.4142135624 |
5 | 1.4142135624 |
```

5.4. Evaluation de la méthode de la sécante

5.4.1. Avantages de la méthode de la sécante

Les principaux avantages de la méthode de la sécante sont :

1. Les hypothèses de départ sont peu contraignantes si la fonction présente un seul zéro.
2. La convergence est rapide dans le cas des racines simples, même si elles sont nombreuses, pour autant qu'on puisse trouver de bons précurseurs x_1 et x_2 .
3. Nécessite une seule évaluation de fonction à chaque étape

5.4.2. Inconvénients de la méthode de la sécante

Le principal inconvénients de la méthode de la sécante est:

1. Comme la méthode de Newton, peut diverger si les données initiales sont mal choisies
2. La lenteur de sa convergence et son manque de précision relatif lorsqu'on a affaire à une racine multiple.
3. Le calcul de $f(x_n) - f(x_{n-1})$ peut produire des erreurs de chute.

Le programme qui suit et les résultats qu'il produit illustrent les inconvénients 2 et 3.

In [117]:

```
% pylab inline
```

Populating the interactive namespace from numpy and matplotlib

In [118]:

```
def f(x):  
    return 16*(sin(x)*cos(x))**2-4*sin(2*x)+1
```

In [120]:

```
secante(1.7, 1.5,f, epsilon = 10**(-15),maxiter = 100)
```

```

1 | 1.4417280513 |
2 | 1.3910826476 |
3 | 1.3617140910 |
4 | 1.3420322608 |
5 | 1.3297147724 |
6 | 1.3218998760 |
7 | 1.3170168170 |
8 | 1.3139699632 |
9 | 1.3120771771 |
10 | 1.3109031568 |
11 | 1.3101760367 |
12 | 1.3097260370 |
13 | 1.3094476913 |
14 | 1.3092755744 |
15 | 1.3091691663 |
16 | 1.3091033894 |
17 | 1.3090627321 |
18 | 1.3090376025 |
19 | 1.3090220709 |
20 | 1.3090124715 |
21 | 1.3090065387 |
22 | 1.3090028720 |
23 | 1.3090006058 |
24 | 1.3089992052 |
25 | 1.3089983396 |
26 | 1.3089978046 |
27 | 1.3089974740 |
28 | 1.3089972696 |
29 | 1.3089971433 |
30 | 1.3089970653 |
31 | 1.3089970171 |
32 | 1.3089969873 |
33 | 1.3089969685 |
34 | 1.3089969571 |
35 | 1.3089969509 |
36 | 1.3089969460 |
37 | 1.3089969435 |
38 | 1.3089969337 |
39 | inf |
```

```

C:\ProgramData\Anaconda3\lib\site-packages\ipykernel_launcher.py:13: Runtime
Warning: divide by zero encountered in double_scalars
del sys.path[0]
```

```

C:\ProgramData\Anaconda3\lib\site-packages\ipykernel_launcher.py:2: RuntimeW
arning: invalid value encountered in sin
```

```

C:\ProgramData\Anaconda3\lib\site-packages\ipykernel_launcher.py:2: RuntimeW
arning: invalid value encountered in cos
```

On remarque que le dernier terme de la suite devient infinie "*inf*". Ceci est dus au fait que la différence $f(x_i) - f(x_{i-1})$ devient très proche de 0 (on divise par zéro).

5.5. Fonction "secante" prédéfinie dans Python

In [122]:

```
from scipy.optimize import *  
secant(g, 1.4, 1.5, tol=0.000000001)
```

Out[122]:

1.5

In [50]:

```
from scipy.optimize import *  
bisect(f,0,2)
```

Out[50]:

1.4142135623715149

In []:

In [2]:

```

from scipy import misc
import numpy as np
import matplotlib.pyplot as plt

def ma_fonction(x):
    y = x*x + 3*x - 5
    return (y)

def ma_derivee(x):
    y = 2*x + 3
    return (y)

def TraceDroiteVerticale(x1, y1, ma_couleur):

    tab_x = [x1 , x1]
    tab_y = [0 , y1]
    plt.plot(tab_x, tab_y, color=ma_couleur)

def TraceDroite(x1, x2, a, b, ma_couleur):

    tab_x = np.linspace(x1,x2,100)
    tab_y = a * tab_x + b
    plt.plot(tab_x, tab_y, color=ma_couleur)

def TraceTangente(x1, x2, derive_en_x1, y1, ma_couleur):

    a = derive_en_x1
    b = -derive_en_x1*x1 + y1
    TraceDroite(x1, x2, a, b, ma_couleur)

def TraceFonction():
    print('Trace fonction')
    tab_x = np.linspace(-6,6,5000)
    tab_y = ma_fonction(tab_x)

    plt.figure()
    plt.plot(tab_x,tab_y, color='red')
    plt.grid()

def AfficherTexte(x, n):
    texte = 'x%s=%f' % (str(n), x)
    plt.text(x, 0, texte )

def NewtonsMethod( x, epsilon=0.000001):

    TraceFonction()

    n = 0
    dif = 2 * epsilon

    while dif > epsilon :

        y = ma_fonction(x)
        y_prime = ma_derivee(x)
        xn = x - y / y_prime
        dif = abs(xn - x)

```

```

TraceDroiteVerticale(x, y, 'green')
TraceTangente(x, xn, y_prime, y, 'green')
#AfficherTexte(x, n)
x      = xn
n      = n+1

return (x, n)

# -----
# Programme principal
# -----
x      = float(input("Quelle est la valeur de départ ? : "))

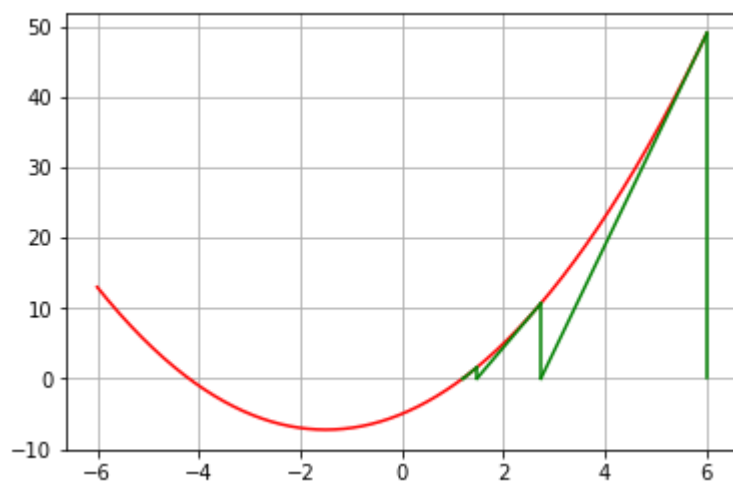
(x,n) = NewtonsMethod(x)

print('x=%f n=%d' % (x, n) )

plt.show()

```

Quelle est la valeur de départ ? : 6
 Trace fonction
 x=1.192582 n=6



In [54]:

```
secant(f, 1.4, 1.5, tol=0.000000001)
```

Out[54]:

1.5

In []:

```
brentq(f, a, b[, args, xtol, rtol, maxiter, ...]) Find a root of a function in a bracketing interval.
brenth(f, a, b[, args, xtol, rtol, maxiter, ...]) Find root of f in [a,b].
ridder(f, a, b[, args, xtol, rtol, maxiter, ...]) Find a root of a function in an interval.
bisect(f, a, b[, args, xtol, rtol, maxiter, ...]) Find root of a function within an interval.
newton(func, x0[, fprime, args, tol, ...]) Find a zero using the Newton-Raphson or secant method.
Fixed point finding:
```

```
fixed_point(func, x0[, args, xtol, maxiter, ...]) Find a fixed point of the function.
```

Multidimensional

General nonlinear solvers:

```
root(fun, x0[, args, method, jac, tol, ...]) Find a root of a vector function.
```

```
fsolve(func, x0[, args, fprime, ...]) Find the roots of a function.
```

```
broyden1(F, xin[, iter, alpha, ...]) Find a root of a function, using Broyden's first Jacobian rank-one update.
```

```
broyden2(F, xin[, iter, alpha, ...]) Find a root of a function, using Broyden's second Jacobian rank-two update.
```